Sri Lanka Institute of Information Technology

IT2080 – IT Project

Year 2 Semester 2 – 2025

Progress Report

Group: **ITP25_B8_184**

- **Title of the project**: NITF Smart Appointment and Queue Management System (ServSync)

Group Details

IT Number	Student Name	Student E-mail Address	Contact Number
IT23638068	SHAVEEN K. K. I.	it236380680@my.sliit.lk	0762606950
IT23768758	VIDANAPATHIRANA I.D	it23768758@my.sliit.lk	0761859187
IT23540026	KALUBOWILAGE S.S	it23540026@my.sliit.lk	0763837189
IT23846036	M.M.A.A AHAMED	it23846036@my.sliit.lk	0758874416
IT23675148	BANDARA H.R.T.L	it23675148@my.sliit.lk	0762243832

Table of Contents

01.System Progress (User Stories-wise) 4
1. User Role & Management 4
2. Appointment Management 5
3. Session & Queue Management 6
4. Customer Feedback & Complaints 8
5. Claims & Insurance Management 9
02.Repositiry link 9
03. ER Diagram 10
04. Normalized Schema 11
05. Test case design 13
06. Network design 18
6.1 Network Topology 18
6.2 Testing & Optimization 18
6.3 Data Flow & Security 19
6.4 Scalability 19
Network design diagram 20
07. High level system design diagram 21
08. Innovative parts 22
09. Commercialization 24
10. Individual Progress 25
10.1 IT23638068– Shaveen K.K.I 25
10.2 IT23540026 – Kalubowilage S.S 34
10.3. IT23540026 – Ahamed M.M.A.A 41
10.4 IT23540026 – Bandara H.R.T.L 46
10.5.IT23540026 – Vidanapathirana I.D 51

01.System Progress (User Stories-wise)

This section illustrates the progress of the functionalities

1. User Role & Management

- Kalubowilage S.S

This module is the foundation of the system, ensuring secure access for all user roles.

No.	Task ID	User Story	User Role	Status
1	USR-01	As a user, I want to register for an account so I can access the system's features.	All Users	Completed
2	USR-02	As a user, I want to log in securely to protect my personal information.	All Users	Completed
3	USR-03	As a user, I want to be able to reset my password if I forget it, so I can regain access to my account.	All Users	To be completed
4	USR-04	As an admin, I want to manage user accounts (create, edit, delete) to control system access.	Admin	Completed
5	USR-05	As an admin, I want to assign roles to staff to define their permissions and access levels.	Admin	Completed
6	USR-06	As a user, I want to view and edit my profile information to keep my details up-to-date.	All Users	Completed

2. Appointment Management

– Ahamed M.M.A.A

This is a core module for customers and reception staff.

No.	Task ID	User Story	User Role	Status
7	APP-01	As a customer, I want to view available appointment slots so I can choose a convenient time.	Customer	Completed
8	APP-02	As a customer, I want to book a new appointment to schedule a visit.	Customer	Completed
9	APP-03	As a customer, I want to view my upcoming and past appointments to keep track of my visits.	Customer	Completed
10	APP-04	As a customer, I want to cancel or reschedule an existing appointment to manage my schedule.	Customer	Completed
11	APP-05	As a receptionist, I want to view the daily appointment schedule to manage customer arrivals.	Receptionist	Completed
12	APP-06	As a receptionist, I want to book, reschedule, or cancel appointments for customers to assist them.	Receptionist	Completed
13	APP-07	As a manager, I want to view appointment statistics and reports to monitor system performance.	Manager	To be completed

3. Session & Queue Management

– Shaveen K.K.I

This module streamlines the on-site customer experience, from the administrative creation of appointment sessions to the real-time management of the customer queue by staff.

No.	Task ID	User Story	User Role	Status
Admin: Session Creation				
14	SESS-01	As an admin, I want to create daily appointment sessions by selecting a branch, counter, and service type, so that I can define staff availability for customer bookings.	Admin	To be completed
15	SESS-02	As an admin, I want to use a template to automatically generate a standard schedule of slots (e.g., 45-min slots with 15-min breaks) to speed up the session creation process.	Admin	To be completed
16	SESS-03	As an admin, I want to customize the generated schedule in a grid view by enabling/disabling slots, adjusting start/end times, and setting the capacity for each slot.	Admin	To be completed
17	SESS-04	As an admin, I want the system to perform validations (e.g., prevent overlapping times, ensure start time is before end time) to ensure the schedule is accurate and reliable.	Admin	To be completed
Customer: Viewing & Joining Queue				
18	QUE-01	As a customer, I want to see the available slots generated from these sessions when I am booking my appointment.	Customer	Completed
19	QUE-02	As a customer with an appointment, I want to get a queue number upon arrival so I can be served in an orderly fashion.	Customer	Completed

20	QUE-03	As a customer, I want to see the real-time queue status on a public display screen so I know my position and the estimated wait time.	Customer	Completed
21	QUE-04	As a customer, I want to receive an SMS notification when my turn is approaching so I don't miss my appointment.	Customer	Completed
Customer Care Officer: Managing Session & Queue				
22	SESS-05	As a Customer Care Officer, I want a real-time dashboard to view my active session, including the current customer's details and the time left in their slot.	C.C. Officer	To be completed
23	QUE-05	As a Customer Care Officer, I want to see a list of all customers waiting in the queue for my session so I can manage the workflow.	C.C. Officer	To be completed
24	QUE-06	As a Customer Care Officer, I want to be able to call the next customer, mark a service as "complete", or handle a "no-show" to control the queue progression.	C.C. Officer	To be completed
Manager: Monitoring				
25	QUE-07	As a manager, I want to view real-time queue data and analytics to identify bottlenecks and monitor staff efficiency.	Manager	To be completed

4. Customer Feedback & Complaints

– Vidanapathirana I.D

This module is crucial for improving service quality.

No.	Task ID	User Story	User Role	Status
26	FB-01	As a customer, I want to submit feedback or a complaint about my experience.	Customer	Completed
27	FB-02	As a customer, I want to track the status of my complaint to know it's being addressed.	Customer	Completed
28	FB-03	As a C.C. Officer, I want to view and manage incoming complaints to resolve customer issues.	C.C. Officer	Completed
29	FB-04	As a C.C. Officer, I want to update a complaint's status and communicate with the customer.	C.C. Officer	Completed
30	FB-05	As a manager, I want to view reports on customer feedback and complaints to identify areas for improvement.	Manager	To be completed

5. Claims & Insurance Management

- Bandara H.R.T.L

This module is a key business function for NITF.

No.	Task ID	User Story	User Role	Status
31	CLM-01	As a customer, I want to submit a new insurance claim online for a quick and easy process.	Customer	Completed
32	CLM-02	As a customer, I want to upload supporting documents for my claim to provide all necessary information.	Customer	Completed
33	CLM-03	As a customer, I want to track the status of my insurance claim to know its progress.	Customer	Completed
34	CLM-04	As a C.C. Officer, I want to review and process incoming claims in a timely manner.	C.C. Officer	Completed
35	CLM-05	As a manager, I want to view claim statistics and reports to monitor the claims department's performance.	Manager	Completed
36	CLM-06	As a manager, I want to approve or reject claims based on the C.C. Officer's assessment.	Manager	To be completed

02.Repositiry link

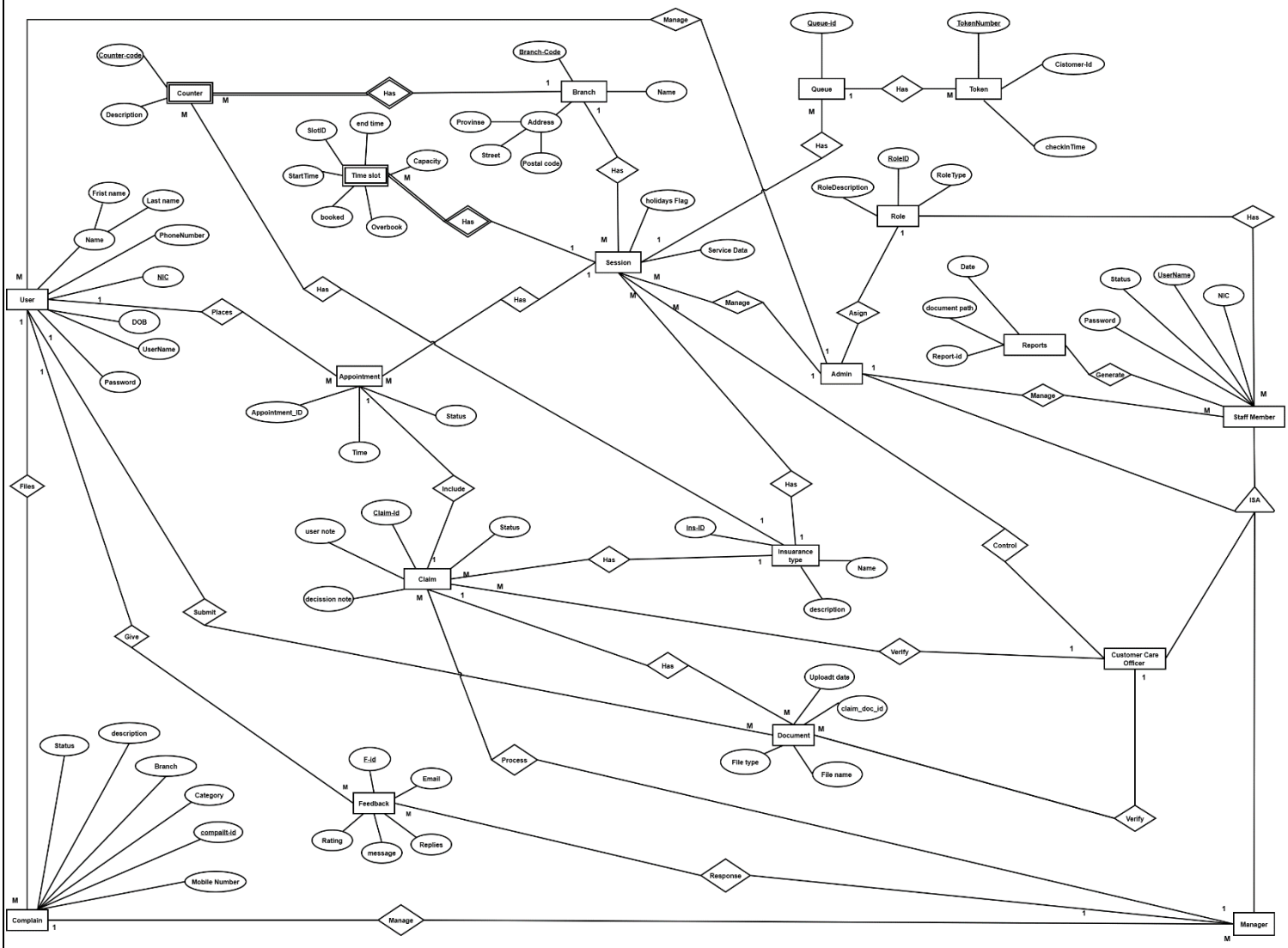
The complete source code for our project, 'ServSync', is hosted on GitHub and managed using Git version control. All development progress and contributions are tracked in this repository.

- **Repository Link:** https://github.com/irushashaveen/ServSync.git

03. ER Diagram

The following Entity-Relationship (ER) Diagram illustrates the database structure for the 'ServSync' system. It outlines the main entities such as

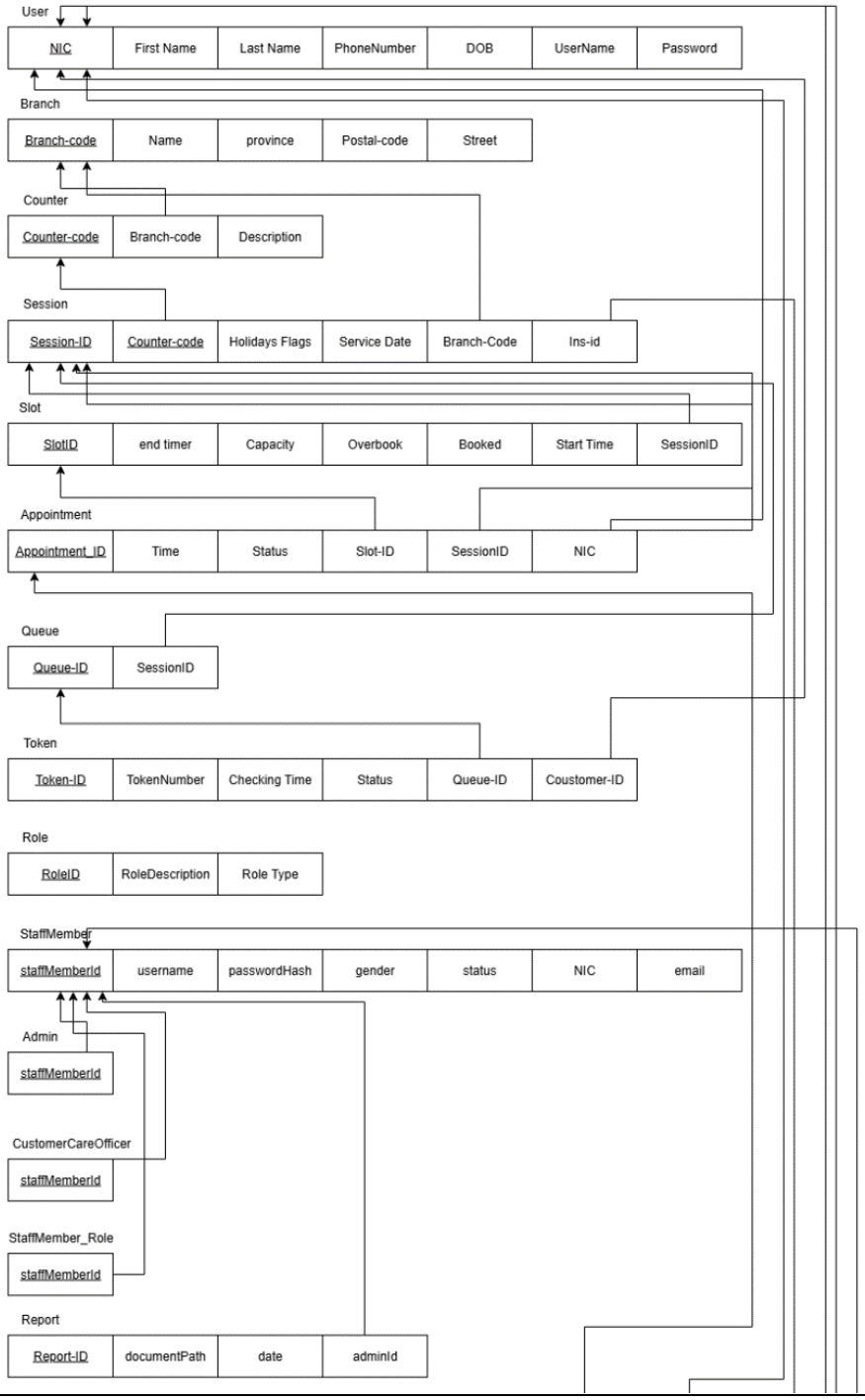
Users, **Appointments**, **Sessions**, and **Claims**, along with the relationships between them.

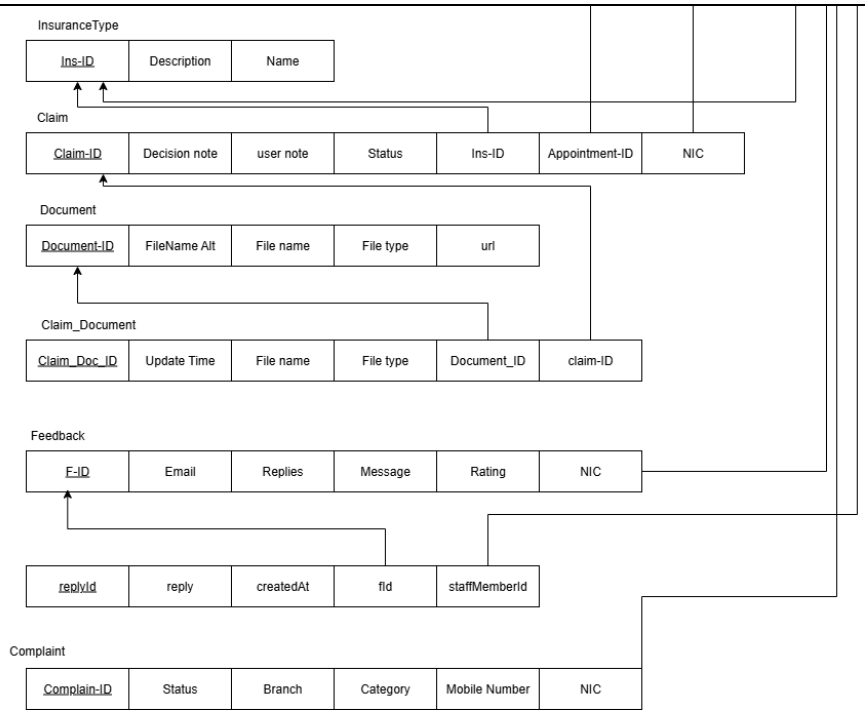


This is our original image : [Original image for better quality click here]

04. Normalized Schema

Derived from the ER Diagram, the following schema presents the final database tables after normalization. The design adheres to the Third Normal Form (3NF) to reduce data redundancy and improve data integrity, ensuring a robust and efficient database.





For a higher-resolution and more detailed view of the schema, the original draw.io diagram can be accessed via the following link [Click to see higher-resolution and more detailed view]

05. Test case design

Test Case ID	Test Scenario	Precondition	Test Steps	Expected Result	Actual Result	Status
SS-TC-001	Customer Login	Customer account exists	1. Open login page 2. Enter valid credentials 3. Click Login	Redirected to Customer Home/Dashboard	As expected	Pass
SS-TC-002	Admin Login (RBAC)	Admin account exists	1. Open login page 2. Enter valid admin credentials 3. Click Login	Redirect to Admin Dashboard with admin menus visible	As expected	Pass
SS-TC-003	Receptionist Login (RBAC)	Receptionist account exists	1. Login as Receptionist	Redirect to Receptionist Dashboard; queue tools visible	As expected	Pass
SS-TC-004	CCO Login (RBAC)	CCO account exists	1. Login as Customer Care Officer	Redirect to CCO Session Dashboard; assigned cases visible	As expected	Pass
SS-TC-005	Manager Login (RBAC)	Manager account exists	1. Login as Manager	Manager analytics/reports menu visible	As expected	Pass
SS-TC-006	Register New User	System allows registration	1. Open Register 2. Fill details 3. Submit	New user created; verification/activation per policy	As expected	Pass
SS-TC-007	Forgot Password	User email exists	1. Click Forgot password 2. Enter email 3. Submit	Reset link/OTP sent; user can set new password	As expected	Pass
SS-TC-008	Admin: Create User & Assign Role	Admin logged in	1. Open User Management 2. Add User 3. Fill details 4. Assign role(s) 5. Save	User created; RBAC stored; audit log written	As expected	Pass

SS-TC-009	Admin: Restrict Privilege Escalation	Admin logged in	1. Try assigning Admin role without elevation rights	Action blocked with insufficient privilege message	As expected	Pass
SS-TC-010	View Available Appointment Slots	Customer logged in; slots published	1. Open Book Appointment 2. Select branch/service	Real-time slots visible (date/time)	As expected	Pass
SS-TC-011	Book Appointment with Docs & QR	Customer logged in; required fields prepared	1. Choose slot 2. Enter claim details 3. Upload docs 4. Confirm	Appointment created; QR + confirmation issued	As expected	Pass
SS-TC-012	Slot Full – Suggest Next Slot	Slots for chosen time are full	1. Select full slot 2. Attempt confirm	System suggests nearest available slot	As expected	Pass
SS-TC-013	Document Validation on Booking	Customer uploads invalid/missing doc	1. Upload wrong type/omit required doc 2. Submit	Clear error prompts; cannot proceed until fixed	As expected	Pass
SS-TC-014	Reschedule Appointment	Appointment exists, future date	1. Open My Appointments 2. Choose booking 3. Reschedule	Slot updated; notification sent; QR updated if needed	As expected	Pass
SS-TC-015	Cancel Appointment	Appointment exists, cancellable	1. Open booking 2. Cancel	Status → Cancelled; notifications sent	As expected	Pass
SS-TC-016	Receptionist QR Check-In	Booking for today	1. Receptionist opens Check-In 2. Scan QR	Status → Checked-in; token generated & queued	As expected	Pass
SS-TC-017	Manual Lookup if QR Unreadable	Booking exists	1. Fail QR scan 2. Search by NIC/Phone/Booking ID	Booking found; proceed to check-in	As expected	Pass
SS-TC-018	Prevent Duplicate Check-In	Booking already checked-in	1. Scan same QR again	Warning; duplicate prevented	As expected	Pass
SS-TC-019	Token Routing to Service Queue	Checked-in completed	1. Complete check-in	Token added to correct service queue; ETA displayed	As expected	Pass

SS-TC-020	Real-Time Queue Display	Display screen online; active queue	1. Serve/pause/transfer tokens 2. Observe display	Display updates within target latency (≤1s)	As expected	Pass
SS-TC-021	Receptionist Manage Queue	Receptionist logged in; active queue	1. Call next 2. Transfer token 3. Pause/resume	Queue state updates; events broadcast to clients	As expected	Pass
SS-TC-022	Notify Customer 'Near Turn'	Notifications enabled	1. Advance queue to N-1 2. Observe customer notification	Timely push/SMS/email as configured	As expected	Pass
SS-TC-023	CCO: Open Assigned Session	Token assigned to CCO	1. CCO opens Session Dashboard 2. Select token	Customer card, timers, notes area visible	As expected	Pass
SS-TC-024	CCO: Add Service Notes	Active session	1. Add internal remarks 2. Save	Notes saved; audit logged; visible only per role	As expected	Pass
SS-TC-025	Submit Insurance Claim (Portal)	Customer logged in	1. Open Claims 2. Fill form 3. Upload docs 4. Submit	Claim created; status Submitted; tracking ID	As expected	Pass
SS-TC-026	Track Claim Status	Claim exists	1. Open My Claims 2. View status	Current status shown with timeline	As expected	Pass
SS-TC-027	CCO: Review Uploaded Docs	Claim assigned; docs uploaded	1. Open claim 2. Review completeness/validity	Missing/invalid flagged; request additional docs	As expected	Pass
SS-TC-028	Verify Bank Details	Validated docs	1. Open bank info 2. Verify per rules 3. Save	Bank details verified and recorded	As expected	Pass
SS-TC-029	Approve & Forward to Manager	Docs & bank verified	1. Click Approve & Forward	Claim moves to Manager review; notifications/logs	As expected	Pass

SS-TC-030	Manager: Approve/Reject Claim	Claim in Manager queue	1. Review package 2. Approve or Reject	Status updates; if reject—reason logged & notified	As expected	Pass
SS-TC-031	Payment Confirmation Flow	Manager approved	1. Trigger payment 2. Confirm success	Payment recorded; customer confirmation sent	As expected	Pass
SS-TC-032	Generate Claim Summary Report	Claim processed	1. CCO generate report 2. Submit to Manager	Report created & stored; visible to Manager	As expected	Pass
SS-TC-033	Feedback Submission	Session completed	1. Open Feedback 2. Select rating/category 3. Submit	Feedback saved; link via SMS works	As expected	Pass
SS-TC-034	Manager: Review & Categorize Feedback	Feedback exists	1. Open Feedback Management 2. Filter 3. Categorize	Category saved; optional assignment for follow-up	As expected	Pass
SS-TC-035	Manager Reports (Queues/Appointments/Claims/Feedback)	Data exists	1. Open Reports 2. Select period/module 3. Generate	Report renders; totals match dashboard	As expected	Pass
SS-TC-036	Export Reports (PDF/Excel)	Report generated	1. Click Export 2. Choose format	File downloads successfully	As expected	Pass
SS-TC-037	Localization (SI/TA/EN)	Locales configured	1. Switch language 2. Navigate key pages	UI text switches; layouts remain intact (RTL-safe)	As expected	Pass
SS-TC-038	Access Control (RBAC)	Multiple roles exist	1. Try accessing restricted module (e.g., CCO → Admin)	Access denied with friendly message	As expected	Pass
SS-TC-039	Audit Logging for Sensitive Actions	Logging enabled	1. Create user 2. Approve claim 3. Change role	Audit entries created (who/what/when/where)	As expected	Pass
SS-TC-040	Performance: Slot Search ≤2s	Published slots	1. Load slot list repeatedly	Response ≤ target; otherwise surface warning	As expected	Pass

SS-TC-041	Performance: Queue Push ≤1s	Active queue	1. Call next token 2. Observe clients	Client UIs update within target latency	As expected	Pass
SS-TC-042	Check-In Reliability (Offline edge)	Flaky network (simulated)	1. Scan QR during brief disconnect	Local cache/auto-sync prevents data loss	As expected	Pass
SS-TC-043	Security: Duplicate Sessions/CSRF/XSS basic	Standard browser	1. Attempt duplicate form submit 2. Inject script in input	Idempotency enforced; inputs sanitized; CSRF protected	As expected	Pass
SS-TC-044	Notifications Delivery & Retry	Gateway configured	1. Trigger SMS/email 2. Simulate failure	Retries/backoff logged; in-app fallback QR available	As expected	Pass
SS-TC-045	(Happy Path) Admin Creates Session with Template	Admin is logged in.	1. Navigate to "Create Session". 2. Select a Branch, Counter, and a future Date. 3. Click "Generate Slots" using the default template. 4. Click "Create Session".	A valid list of non-overlapping slots is generated. The session is successfully created and saved in the database.	As expected	Pass
SS-TC-046	(Validation) Attempt to Create Session in the Past	Admin is logged in.	1. Navigate to "Create Session". 2. Select a date that is in the past. 3. Attempt to generate slots or create the session.	An error message is displayed (e.g., "Service date must be in the future"). The system prevents the creation of the session.	As expected	Pass
SS-TC-047	(Validation) Manually Create Overlapping Slots	Admin is logged in and has generated a list of slots.	1. Manually edit the end time of one slot to be after the start time of the next slot. 2. Click "Create Session".	A validation error is displayed (e.g., "Overlap between slots detected"). The session is not created.	As expected	Pass
SS-TC-048	(UI Logic) Dynamic Counter Filtering	Admin is logged in on the "Create Session" page.	1. Select a Branch from the dropdown. 2. Observe the options in the "Counter" dropdown. 3. Change the selected Branch to a different one.	The "Counter" dropdown automatically updates to show only the counters associated with the newly selected branch.	As expected	Pass

06. Network design

6.1 Network Topology

The ServSync system adopts a **Star Topology**, where all user devices and servers connect through a central switch/router. This provides reliability, scalability, and simplified management.

Components:

- **User Devices:** Customers (PC/Mobile), Staff PCs (Admin, Manager, Receptionist, CCO).
- **Backend Servers:** Host Node.js/Express API, authentication, and queue logic.
- **Database Server:** MongoDB instance for appointments, users, claims, and logs.
- **IoT Devices:** ESP32 with LCD displays showing live queue status at branch counters.

IP Scheme:

- Network: 192.168.1.0/24
- Range: 192.168.1.1 – 192.168.1.254
- Subnet: 255.255.255.0

Backup & Redundancy:

- **Backend Servers:** Load-balanced across multiple instances for high availability.
- **Database Server:** Replica set ensures backup and failover in case of primary failure.
- **IoT Devices:** Local cache mode in case of temporary disconnection.

6.2 Testing & Optimization

- **Security Testing:** Run penetration testing for login/authentication, SQL/NoSQL injection checks, and session hijacking prevention.
- **Performance Testing:** Queue response time < 500ms; booking confirmation < 2s.
- **Optimization:** API Gateway or reverse proxy (e.g., Nginx) for routing frontend requests and managing traffic efficiently.

Architecture (MERN):

- **Frontend (React.js):** Role-based dashboards, customer booking interface.

- **Backend (Node.js/Express):** Appointment, queue, claims, notifications, role-based access.
- **Database (MongoDB):** Users, roles, sessions, claims, audit logs.
- **(Optional) Redis Cache:** For real-time queue state and notification broadcasting.

6.3 Data Flow & Security

Flow:

1. Customer/Staff → **Frontend (React)**
2. Requests → **Backend (Express APIs, HTTPS)**
3. Data stored/retrieved from **MongoDB**
4. Queue updates published to **IoT devices** via WebSocket/Redis Pub-Sub

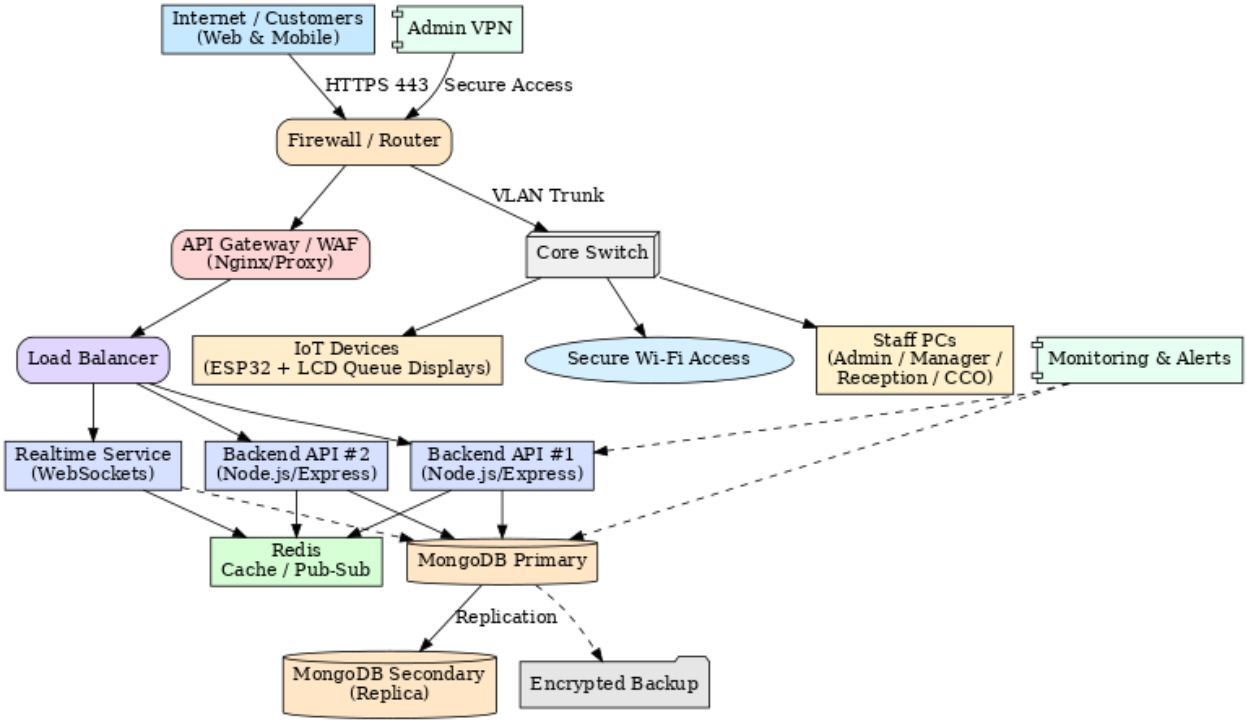
Security Measures:

- **HTTPS Everywhere:** Secure API calls and dashboards.
- **Authentication & Authorization:** JWT tokens with role-based access.
- **Database Security:** MongoDB RBAC, encrypted backups.
- **Network Security:** Firewalls, VPN for admin access, rate limiting.
- **Audit Logging:** Track login attempts, role changes, and claim updates.

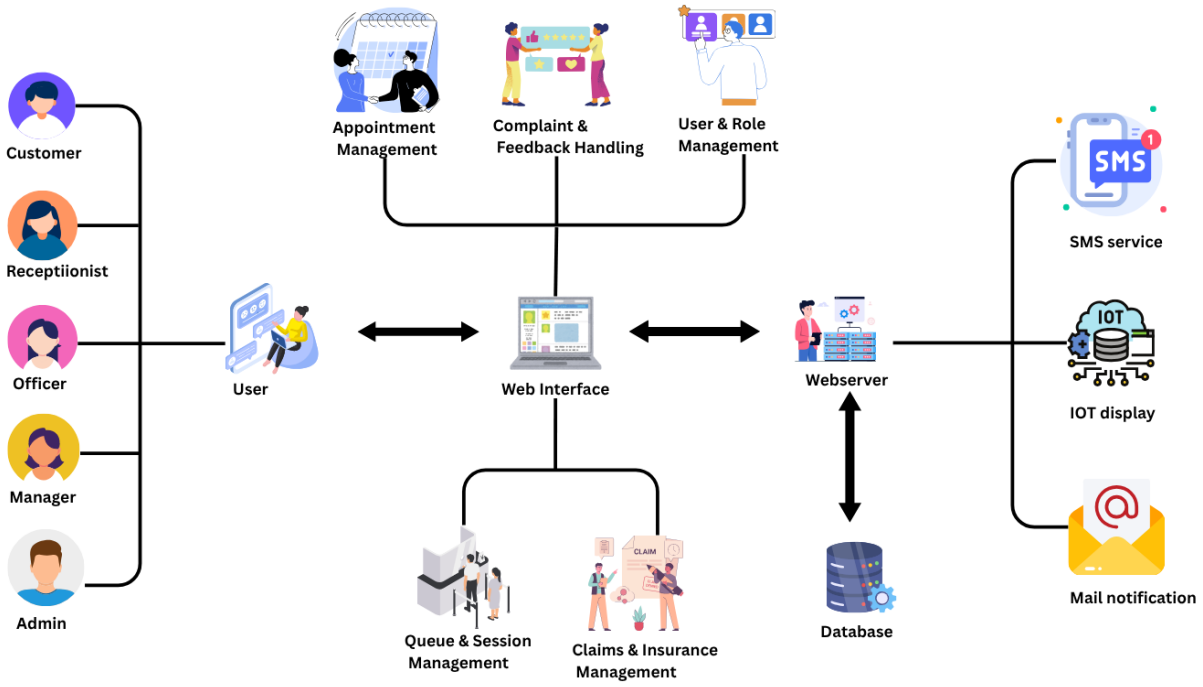
6.4 Scalability

- **Load Balancing:** Distributes API requests across multiple backend servers.
- **Cloud Deployment:** Containerized deployment (Docker) for flexibility; can scale horizontally during peak times.
- **IoT Scalability:** Multiple ESP32 displays supported per branch.
- **Database Scaling:** Sharding or read replicas for handling increased traffic.

Network design diagram



07. High level system design diagram



08. Innovative parts

- **Real-Time Queue & Session Management**
  ServSync introduces a real-time digital queue management system that replaces manual token distribution. Customers receive unique digital tokens and can monitor their queue status through IoT-powered displays (ESP32 + LCD) and dashboards. Customer care officers have tools to mark sessions as completed, delayed, or missed, with the system instantly updating to the next customer. This ensures smooth service flow, reduces congestion, and minimizes waiting times.

- **QR Code–Based Smart Check-In**
  A unique QR code is generated for each appointment, enabling customers to check in instantly at the reception desk. The QR-based system prevents duplicate entries, speeds up the verification process, and automatically updates queue positions in real time. This innovation eliminates long lines at counters and enhances the overall check-in experience.

- **Integrated Document Upload with Appointments**
  Unlike other systems, ServSync allows customers to upload claim-related documents (medical bills, accident reports, or IDs) during the booking process. These documents are securely stored and linked with the appointment or claim ID, ensuring that officers have the required files beforehand. This reduces delays, prevents repeat visits due to missing documents, and streamlines the claims process.

- **Elderly-Friendly and Multilingual Interface**
  To ensure inclusivity, ServSync provides an interface with larger fonts, guided navigation, and simplified steps for senior citizens. The platform supports Sinhala, Tamil, and English, making it accessible to a wide range of users. Compliance with accessibility guidelines (screen reader support and mobile responsiveness) makes the system user-friendly for both tech-savvy and less experienced users.

- **Smart Notifications and Alerts**
  The system integrates SMS, email, and in-app notifications to keep customers informed about appointment confirmations, queue positions, or missing documents. Staff members also receive alerts for urgent cases, no-shows, or system errors. By centralizing communication into one system, ServSync ensures timely updates and reduces the risk of miscommunication.

- **Role-Based Dashboards and Analytics**
  ServSync provides personalized dashboards for each user role, such as admins, managers, receptionists, and officers. Managers can view real-time analytics on queue flow, staff efficiency, and peak hours, while receptionists and officers manage daily tasks more effectively. These dashboards generate reports and insights that support data-driven decision-making.

- **Customer Feedback and Complaint Tracking**
  After each session, customers are prompted to submit feedback through automated SMS or email links. Complaints are tracked with unique IDs, enabling customers to monitor their resolution progress. Managers can review trends in feedback and complaints to identify service gaps and take corrective actions, ensuring continuous quality improvement.
- **Integrated Claims and Insurance Management**
  The system allows customers to initiate and track insurance claims digitally, upload supporting documents, and monitor progress in real time. Officers can verify and process claims, while managers can approve or reject them with complete transparency. This integration directly supports NITF's business operations and ensures efficiency in claims handling.

09. Commercialization

Market Research:
The National Insurance Trust Fund (NITF) receives hundreds of customers daily, but manual queues and claim handling reduce efficiency. There is growing demand for digital service platforms in Sri Lanka's insurance and government sectors. ServSync directly addresses this gap with integrated appointment booking, queue management, and claims processing.

Competitive Analysis:
Existing platforms such as eChannelling and immigration e-services provide only basic scheduling and lack real-time queue tracking, document uploads, and role-based dashboards. ServSync differentiates itself by combining QR-based check-ins, IoT queue displays, claim integration, and smart notifications, offering a comprehensive and scalable solution.

Customer Support & Feedback System:

- Built-in feedback and complaint tracking with unique IDs.
- Automatic feedback requests after each session.
- Future integration of AI-driven chatbots to handle FAQs, improving efficiency and engagement.

Legal Compliance & Sustainability:

- All customer and claim data encrypted at rest and in transit.
- Aligned with Sri Lanka's digital government and data protection policies.
- Scalable architecture to support multiple NITF branches and other organizations.

Deployment & Revenue Model:

- Initial deployment at NITF headquarters, followed by branch expansion.
- Training sessions for staff to ensure smooth adoption.
- Commercialization through **subscription-based SaaS** or **one-time licensing with maintenance contracts**.

Expansion & Growth:
After success at NITF, ServSync can be scaled to other public and private institutions such as hospitals, banks, and government service centers. Future upgrades like AI-powered analytics and multilingual chatbots will support long-term competitiveness.

10. Individual Progress

10.1 IT23638068– Shaveen K.K.I

I was the lead developer for the **Session and Queue Management** module, which is the operational heart of the 'ServSync' system. My responsibilities were divided into two main areas: **Session Creation & Configuration** and **Live Session Control**.

1. Session Creation & Configuration:

- I designed and built an automated, smart UI for **session creation** that simplifies the tedious task of manual scheduling. With just a few clicks, an administrator can generate a full day's schedule.
- I implemented the functionality for admins to **update, edit, and mark entire sessions as holidays**.
- I also developed the administrative features for creating and managing new **branches and counters**, which are the foundation for session scheduling.

2. Live Session Control & Customer Flow:

- Once a customer checks in via QR code, my system takes over by **generating a token and updating the queue in real-time**.
- I created the **Session Control Dashboard** for Customer Care Officers (CCOs), allowing them to start a session, view a customer's appointment and claim details, and manage the customer flow.
- The system pushes live queue status updates to the **IoT displays** at the branch.
- The dashboard includes a "Next Customer" function that helps the CCO manage their time efficiently based on the total session size and calculated time per appointment.
- Upon completion of each appointment, I implemented an automated service that sends an **SMS to the customer, requesting feedback** on their experience.

	Completion level of features	
	Features	Progress
Completed	CRUD operation – Branch & Counter Management	100%
	Admin: Smart UI for Automated Session Creation	100%
	Admin: Session Update, Edit, and Delete functionality	100%

	Real-time Token Generation on Customer QR Check-in	100%
	CCO Dashboard: Live Session Controls (Next, Transfer, End Session)	100%
	Automated Feedback SMS after Session Completion	100%
	Smart Session Control Dashboard for CCO	100%
	CCO Dashboard: Live Session Controls (Next, Transfer, End Session)	100%
To be completed (Working on)	Real-time Queue Updates on CCO Dashboard & IoT Display	50%
	CCO Dashboard: Time Management Calculations & Analytics	40%
	Frontend and Backend validations for CCO Dashboard	70%
	Error Handling for Real-Time Service (e.g., connection loss)	20%
	CCO Dashboard: Time Management Calculations & Analytics	30%

Any SQL Queries Used

No, because MongoDB (via Mongoose) was used as the database, which is a NoSQL database system.

Algorithm

A. Slot Generation (Template)

1. Input: startHM, endHM, slotLen, breakLen, lunchStart, lunchEnd, defaultCapacity, serviceDate.
2. Set cursor = startHM.
3. While cursor + slotLen ≤ endHM:
    - s = cursor, e = cursor + slotLen.
    - If [s,e] overlaps [lunchStart,lunchEnd], set cursor = lunchEnd and continue.
    - Push slot {startHM: s, endHM: e, capacity: defaultCapacity, checked: true}.
    - cursor = e + breakLen.
4. Return slots.

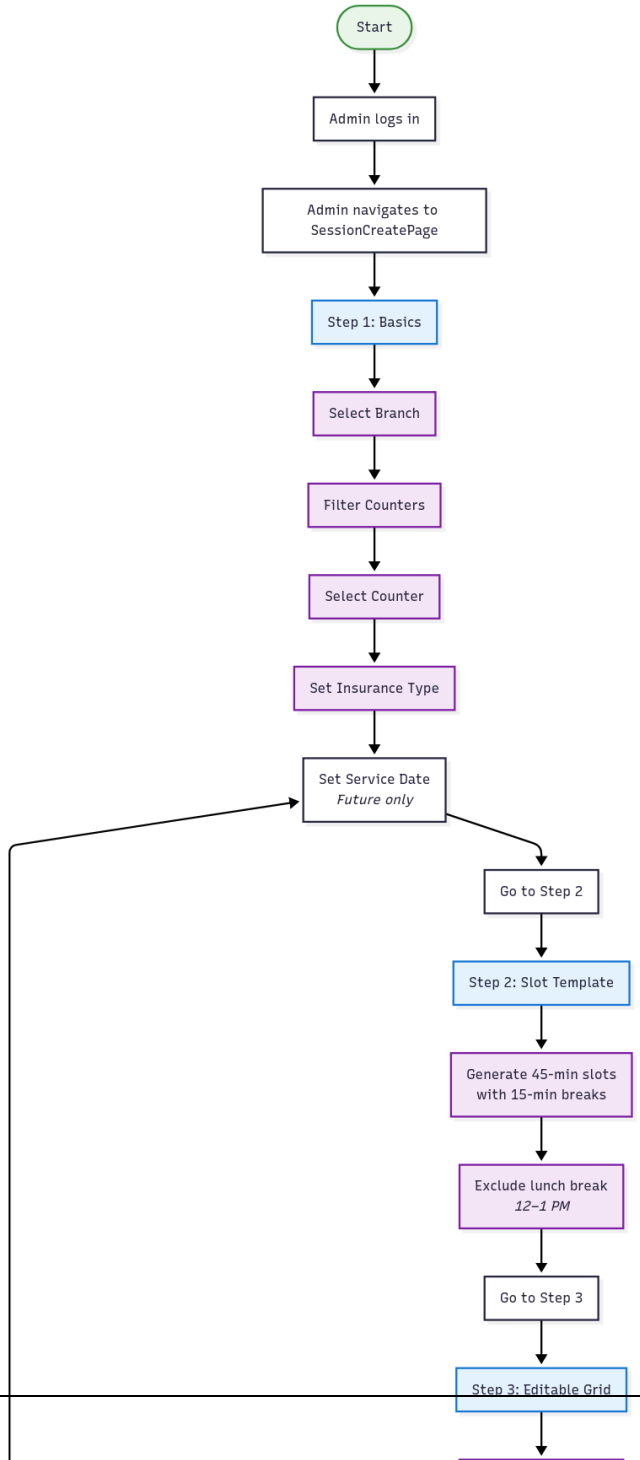
B. Validation

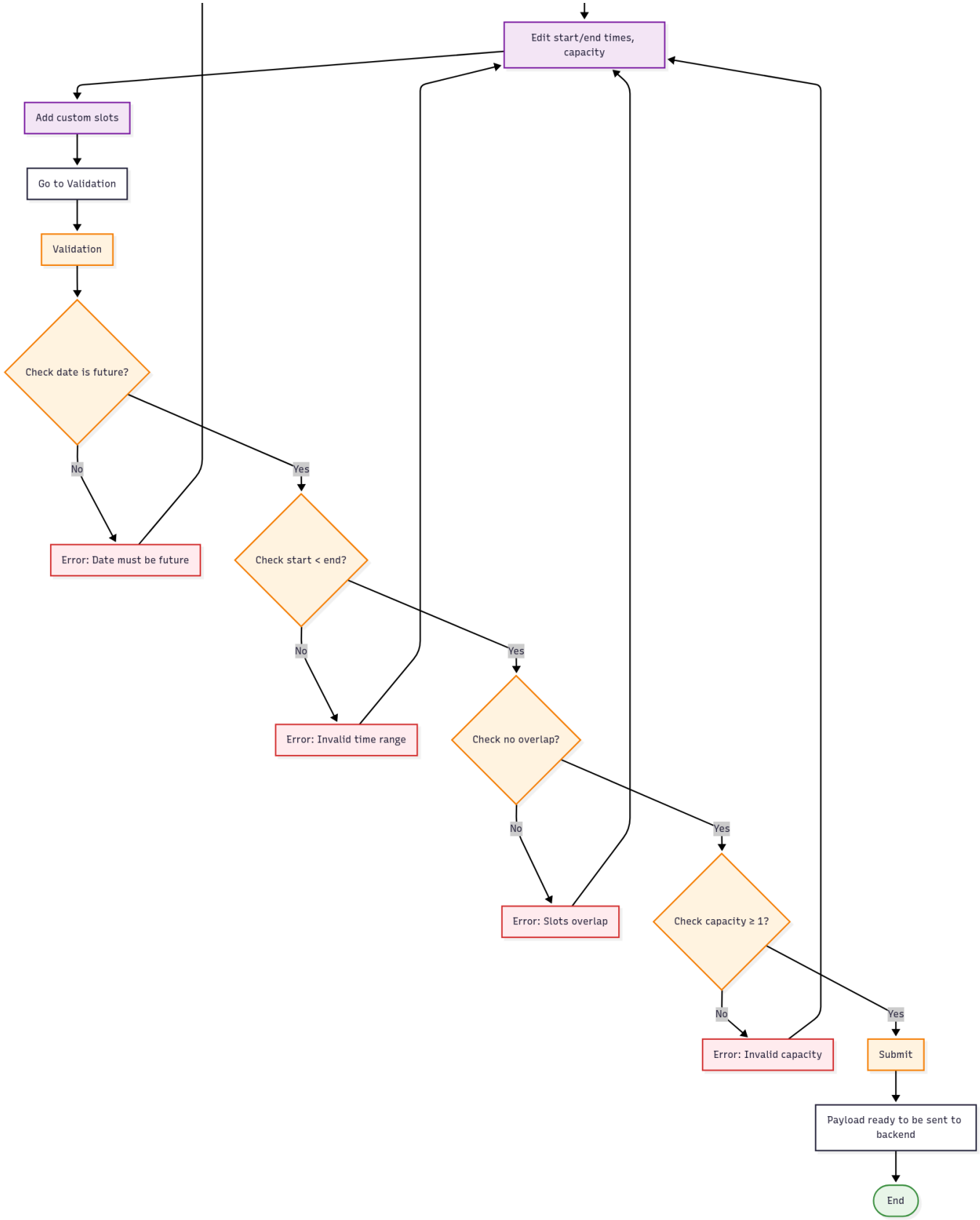
- Date: serviceDate > today.
- For each **checked** slot:
    - start < end.
    - capacity ≥ 1.
    - start and end produce ISO strings falling on the **same UTC day** as serviceDateT00:00Z.
- Overlaps: sort by start; ensure curr.start ≥ prev.end.

C. Payload Transform

- serviceDate = localDateTimeToUTC(serviceDate, "00:00").
- Each slot: startTime = localDateTimeToUTC(serviceDate, startHM); same for endTime.
- Compose Session document fields.

Flow Chart


Start
Admin logs in
Admin navigates to SessionCreatePage
Step 1: Basics
Select Branch
Filter Counters
Select Counter
Set Insurance Type
Set Service Date
Future only
Go to Step 2
Step 2: Slot Template
Generate 45-min slots with 15-min breaks
Exclude lunch break
12–1 PM
Go to Step 3
Step 3: Editable Grid


Edit start/end times, capacity
Add custom slots
Go to Validation
Validation
Check date is future?
No
Yes
Error: Date must be future
Check start < end?
No
Yes
Error: Invalid time range
Check no overlap?
No
Yes
Error: Slots overlap
Check capacity ≥ 1?
No
Yes
Error: Invalid capacity
Submit
Payload ready to be sent to backend
End

```
PROGRAM SessionCreatePage

// Initialization
FUNCTION initializeSessionCreatePage()
    SET branchId, counterId, insuranceTypeId, serviceDate = ""
    SET rows = [], errors = [], tomorrow = getCurrentDate() + 1 day
END FUNCTION

// Step 1: Basics
FUNCTION handleBranchSelection(selectedBranchId)
    SET branchId = selectedBranchId
    SET counterId = ""  // Reset counter
    DISPLAY filtered counters
END FUNCTION

FUNCTION handleCounterSelection(selectedCounterId)
    SET counterId = selectedCounterId
    SET insuranceTypeId = findInsuranceTypeByCounter(selectedCounterId)
END FUNCTION

FUNCTION handleServiceDateSelection(selectedDate)
    IF selectedDate < tomorrow THEN
        DISPLAY error "Date must be in the future"
        RETURN
    END IF
    SET serviceDate = selectedDate
```

```
END FUNCTION

// Step 2: Slot Generation
FUNCTION generateSlots()
    IF serviceDate is empty THEN
        SET errors = ["Pick a service date first."]
        RETURN
    END IF

    SET slots = []
    SET cursor = startTime
    WHILE cursor + slotLength <= endTime DO
        CREATE slot and ADD to slots
        MOVE cursor by slotLength + breakLength
    END WHILE
    UPDATE rows with slots
END FUNCTION

// Step 3: Editable Grid (Slot Management)
FUNCTION handleSlotSelection(slotId, isChecked)
    UPDATE rows[slotId].checked = isChecked
END FUNCTION

FUNCTION addCustomSlot()
    CREATE newSlot and ADD to rows
END FUNCTION

FUNCTION removeSlot(slotId)
```

```
    REMOVE slot from rows
END FUNCTION

// Validation
FUNCTION validateForm()
    SET validationErrors = []
    IF required fields are empty, ADD errors
    IF serviceDate < tomorrow, ADD error
    FOR each slot in rows:
        IF slot time or capacity is invalid, ADD error
    END FOR
    IF any overlaps, ADD error
    SET errors = validationErrors
    RETURN errors is empty
END FUNCTION

// Submit Session
FUNCTION handleSubmit()
    IF NOT validateForm() THEN
        DISPLAY errors
        RETURN
    END IF

    SET payload with session details and slots
    SEND payload to backend API
    DISPLAY success or error response
END FUNCTION
```

```
// Main Flow
FUNCTION main()
    initializeSessionCreatePage()
    DISPLAY Step 1: Branch, Counter, Insurance, Service Date
    WAIT for user input
    DISPLAY Step 2: Slot Configuration and Generation
    WAIT for user input
    DISPLAY Step 3: Editable Grid (Edit, Add, Remove Slots)
    WAIT for user input
    ON submit: CALL handleSubmit()
END FUNCTION

END PROGRAM
```

10.2 IT23540026 – Kalubowilage S.S

The purpose of the User and Role Management module in ServSync is to ensure secure and organized access to the system by defining clear roles and permissions for each user type. This module allows the system to control who can access specific features, ensuring both data protection and efficient workflows across NITF branches. It also provides administrators with the ability to create, update, and deactivate accounts, while maintaining audit trails for accountability.

1. User Account Management

I designed and implemented a **secure, user-friendly system** for managing user accounts, allowing administrators to **create**, **update**, **delete**, and **reset** user credentials. I focused on making the process **streamlined and efficient** by introducing role-based dashboards, where admins can easily access, modify, or deactivate user accounts.

Key functionalities I developed include:

- **User Registration**: A secure registration system with email/phone validation and password hashing using **bcrypt**.
- **Password Management**: Admins have the ability to reset or modify passwords, with **email notifications** sent for confirmation.
- **User Audits**: I incorporated an **audit log system** to track changes made to user accounts, ensuring transparency and security.
- **User Notifications**: Integrated automated notifications (via email/SMS) for **new account creation**, **password resets**, and **role updates** to keep users informed.

2. Role-Based Access Control (RBAC)

I built a **flexible RBAC system** to enforce security across the platform. This allows **admins** to assign roles (e.g., **Admin, Manager, Receptionist, Customer Care Officer**) and assign different **permissions** to each role. This ensures that users only have access to the sections and actions appropriate to their roles.

Key features I developed for RBAC:

- **Role Creation and Assignment**: Admins can create and assign roles to users dynamically, controlling access levels throughout the system.

- **Role Permissions**: Each role has its own **set of permissions**, determining what actions users can perform (e.g., viewing sensitive data, creating new records, etc.).
- **Role Updates**: Admins can **update** roles or **revoke** permissions as needed, ensuring that access remains appropriate to organizational needs.
- **Session Control**: **Session management** is built-in, with **automatic session timeouts** and user logout capabilities for enhanced security.
- **Permissions Enforcement**: Ensured that role permissions are applied consistently across the system, preventing unauthorized access to restricted areas.

Completion Level of the Features

User and Role management	Completion level of features	
	Features	Progress
Completed	CRUD operation – Customer registration and profile management	100%
	CRUD operation – User management(Admin side), Delete, Edit, Update accesses.	100%
	Secure login/logout(JWT authentication + session handling)	100%
	Search options	100%
	Frontend UI implementation	100%
	CRUD operation - Staff management (Admin, Manager, Receptionist, Customer Care Officer)	100%
	Role-based dashboard redirection and other navigations	100%
To be completed (Working on)	Role-Based Access Permissions (RBAC Middleware)	50%
	Password Reset Functionality with email verification	20%
	Frontend and Backend validations	80%
	Audit Logs for User Activity (login, role updates, deletions)	50%

	System analytics report generation	60%

Any SQL Queries Used

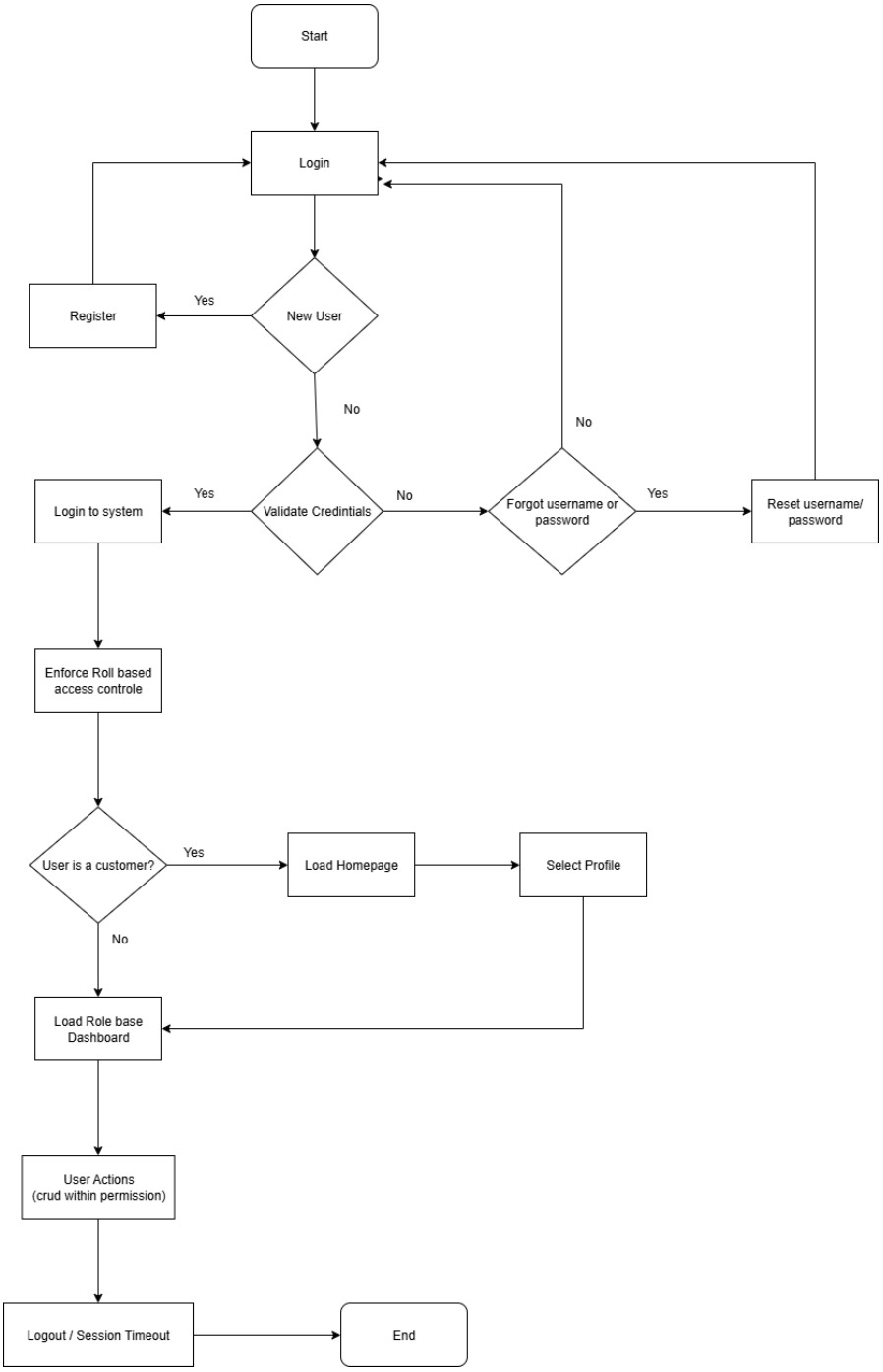
No, because MongoDB (via Mongoose) was used as the database, which is a NoSQL database system.

Algorithm Used

1. Start
    a. The user accesses the system.
2. Login Process:
    a. Check if the user is new (registration).
    b. If the user is new, prompt for registration.
    c. If the user already exists, proceed to validate credentials.
3. Validate Credentials:
    a. Validate entered credentials (email, password).
    b. If the credentials are valid, authenticate and log the user in.
    c. If the credentials are invalid, prompt for re-entry or show an error.
4. Forgot Username/Password:
    a. If the user forgot their credentials, allow them to reset their password.
5. Enforce Role-Based Access Control:
    a. After login, check the user's role (admin, customer, etc.).
    b. Direct the user to the appropriate dashboard or page based on their role.
6. User Actions:
    a. If the user is a customer, load the homepage and allow them to select a profile.
    b. If the user is an admin, load the role-based dashboard and allow user management operations.
7. Logout/Session Timeout:
    a. Automatically log the user out after a session timeout or when they choose to log out.

End

Flow Charts


Start
Login
Register
Yes
New User
No
No
Login to system
Yes
Validate Credintials
No
Forgot username or password
Yes
Reset username/ password
Enforce Roll based access controle
User is a customer?
Yes
Load Homepage
Select Profile
No
Load Role base Dashboard
User Actions (crud within permission)
Logout / Session Timeout
End

Pseudocode

```
// Main Function: User Management Flow
function userManagementFlow():
    // Step 1: Start
    show("Welcome to the System")

    // Step 2: Login Process
    userType = getUserType()

    if userType == 'New':
        registerUser()  // User registers an account
    else:
        loginCredentials = getLoginCredentials()
        validateCredentials(loginCredentials)

    // Step 3: Validate Credentials
    if validateCredentials(loginCredentials) == True:
        // Step 4: Forgot Username/Password
        if forgotCredentials():
            resetPassword()
        else:
            // Step 5: Enforce Role-Based Access Control
            if checkRole(userRole):
                showDashboard(userRole)
            else:
                show("Access Denied")
```

```
        // Step 6: User Actions
        if isCustomer():
            homepage = loadHomepage()
            selectProfile()
        else if isAdmin():
            loadRoleBasedDashboard()

        // Step 7: Logout / Session Timeout
        logout()
    else:
        showError("Invalid Credentials")

// Function to check if user is valid
function validateCredentials(loginCredentials):
    // Logic to validate username and password
    return True or False

// Function to handle registration
function registerUser():
    show("Please fill registration details")
    // Collect user registration details and store

// Function for password reset
function resetPassword():
    show("Please provide your email or username to reset your password")
    // Handle password reset logic

// Function to check the user's role
```

```
function checkRole(userRole):
    if userRole == "admin":
        return True
    else if userRole == "customer":
        return True
    return False

// Function to display dashboards based on role
function showDashboard(userRole):
    if userRole == "admin":
        show("Admin Dashboard")
    else if userRole == "customer":
        show("Customer Homepage")

// Function to handle user logout
function logout():
    show("Logging out...")

// Main Function Call
userManagementFlow()
```

10.3. IT23540026 – Ahamed M.M.A.A

Appointment management

	Completion level of features	
	Features	Progress
Completed	CRUD operation – Service Booking: Users can browse available services and schedule appointments, edit, cancel	100%
	Admin Dashboard – Admins can manage users, service providers, and view booked appointments	100%
	CRUD operation- Service Provider Dashboard – Service providers can manage their service schedules and update availability	100%
	Responsive Design – The app is fully responsive and works across various devices	100%
To be completed (Working on)	Automatic QR Code Generation (downloadable after booking)	50%
	Appointment Notifications & Reminders (Email/SMS)	40%
	Frontend & Backend Validations for Appointment Form	70%
	Appointment Rescheduling Flow Improvements	60%
	System Analytics for Appointment Trends	50%

Any SQL Queries Used

No, because MongoDB (via Mongoose) was used as the database, which is a NoSQL database system.

1 Authentication Algorithm

- Checks whether the user is logged in.
- Validates login credentials (username/password).
- Directs user either forward (if authenticated) or back with error.

2 Service Availability Algorithm

- Queries the system to check if services are active.
- Displays available services if active, otherwise shows “Service unavailable”.

3 Slot Allocation Algorithm

- Checks the database for available slots.
- If slots are full → shows “No slots available”.
- If slots exist → user proceeds to booking.

4 Form Validation Algorithm

- Ensures required fields are completed.
- Detects errors (missing/invalid inputs).
- Provides feedback and loop for correction**.**

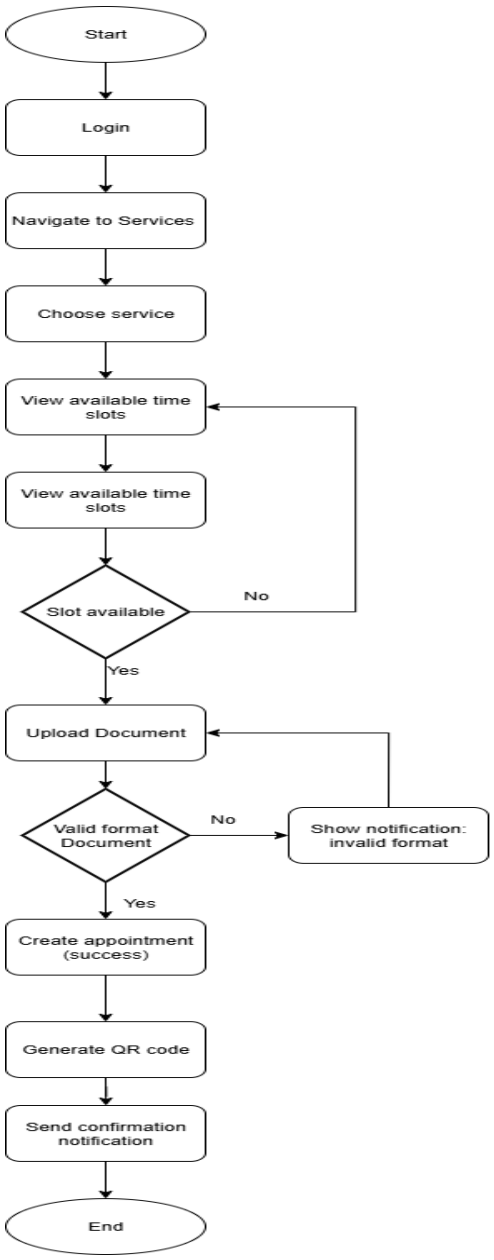
5 Document Verification Algorithm

- **Ensures required files are uploaded.**
- **Validates format (PDF/JPG/PNG).**
- **Rejects invalid documents with resubmission option.**

6 Transaction & Confirmation Algorithm

- Once everything is valid, generates a unique QR code.
- Logs transaction details (user ID, service, slot, timestamp) in the database.
- Sends notification (email/SMS/in-app) to confirm booking.

Flow chart


Start
Login
Navigate to Services
Choose service
View available time slots
View available time slots
Slot available
No
Yes
Upload Document
Valid format Document
No
Show notification: invalid format
Yes
Create appointment (success)
Generate QR code
Send confirmation notification
End

Pseudocode

```
BEGIN

// LOGIN STAGE
DISPLAY "Login Page"
IF user IS NOT logged_in THEN
   PROMPT "Enter Username & Password"
   IF authentication_successful THEN
      CONTINUE
   ELSE
      DISPLAY "Error: Invalid Login"
      GO TO LOGIN
   ENDIF
ENDIF

// SERVICE SELECTION
DISPLAY "Available Services"
IF services_active = FALSE THEN
   DISPLAY "Service Unavailable"
   GO TO SERVICE_SELECTION
ENDIF

// SLOT SELECTION
DISPLAY "Available Time Slots"
IF no_slots_available THEN
   DISPLAY "No slots available, try later"
   GO TO SERVICE_SELECTION
ELSE
```

```
    PROMPT "Select Slot"
ENDIF

// FORM VALIDATION
PROMPT "Fill Appointment Form"
IF form IS NOT valid THEN
    DISPLAY "Form Errors – Please Correct"
    GO TO FORM VALIDATION
ENDIF

// DOCUMENT UPLOAD
PROMPT "Upload Required Documents"
IF documents_missing OR invalid_format THEN
    DISPLAY "Upload Missing/Valid Documents (PDF/JPG/PNG)"
    GO TO DOCUMENT UPLOAD
ENDIF

// BOOKING CONFIRMATION
GENERATE unique_QR_code
LOG transaction(user, service, slot, timestamp, QR_code) INTO database

// NOTIFICATION
SEND notification(user_email/phone, "Booking Confirmed with QR Code")
DISPLAY "Download QR Code"

// END
DISPLAY "Booking Successful"
END
```

10.4 IT23540026 – Bandara H.R.T.L

Claims and Insurance management

The **Claim and Insurance Management** module in the **ServSync** system is responsible for managing the entire lifecycle of insurance claims, from initiation to payment processing. This module allows customers to upload their documents during the appointment phase, after which a Customer Care Officer (CCO) reviews them for completeness and correctness. If necessary, more documents can be requested. Once the documents are approved by the CCO, they are sent to the Manager for further verification. The system also ensures that all steps, including document validation, approval, and payment processing, are tracked and documented for audit purposes.

- **Customer Role**: Upload claim documents, view claim status, and receive updates on approval or rejection.
- **CCO Role**: Review uploaded documents, request additional documents, approve or reject documents.
- **Manager Role**: Verify approved documents, process payments, and confirm the transaction.
- **Admin Role**: Monitor and track the entire claim process and ensure that it aligns with insurance policies.

Claim and Insurance Management	Completion level of features	
	Features	Progress
Completed	Document upload by customer	100%
	Customer Care Officer (CCO) reviews documents	100%
	Request additional documents from the customer	100%
	Approve or reject documents	100%
	Send approved documents to Manager	100%
	Manager verifies documents and processes payment	100%

	Notification sent to the customer upon claim approval/rejection	100%
	Final report preparation by CCO	100%
	Deliver final report to the manager	100%
To be completed (Working on)	Integration with payment gateways for automated payment processing	60%
	System analytics report generation	50%

Any SQL Queries Used

No, because MongoDB (via Mongoose) was used as the database, which is a NoSQL database system.

Algorithm

1. Start
2. Wait until the customer uploads documents during the appointment phase.
3. Retrieve the uploaded documents.
4. Loop (Repeat until documents are approved and payment is made):
   a. Review the uploaded documents by Customer Care Officer (CCO).
   b. If documents are incomplete or invalid:
      i. Request additional documents from the customer.
      ii. Wait for the customer to upload the additional documents.
      iii. Append the newly uploaded documents to the existing documents list.
      iv. Go back to step 4a.
   c. If documents are complete:
      i. Send the approved documents to the Manager for verification.
   d. Manager verifies the documents:
      i. If the documents are correct:
         1. Manager makes the payment.

2. If payment is successful:

a. CCO prepares the final report.

b. Deliver the final report to the Manager.

c. End the process.

3. If payment fails:

a. Log the payment failure and return to step 4a for further action.

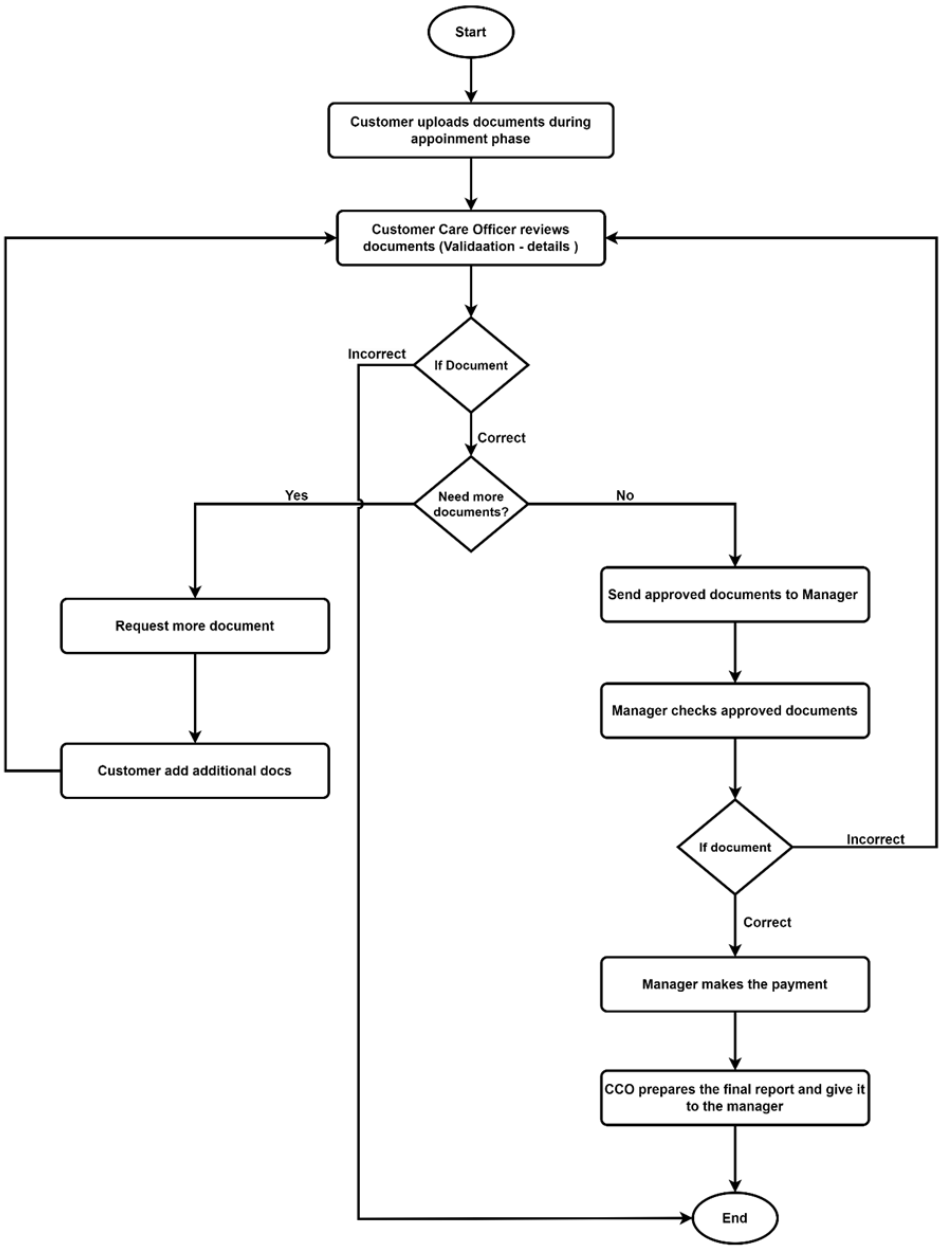
ii. If the documents are incorrect:

1. Provide feedback to the CCO.

2. Return to step 4a for review.

5. End

Flow Chart



Pseudocode

PROCEDURE ProcessAppointmentDocuments(appointmentId) documents ← GET_UPLOADED_DOCUMENTS(appointmentId)

```
WHILE NOT IS_DOCUMENT_APPROVED(documents) DO
    reviewResult ← CCO_REVIEW(documents)

    IF reviewResult.NEED_MORE_DOCUMENTS THEN
```

```
        REQUEST_ADDITIONAL_DOCS(appointmentId)
        documents ← WAIT_FOR_ADDITIONAL_DOCS(appointmentId)
    ELSE
        SEND_TO_MANAGER(documents)
        IF MANAGER_APPROVES(documents) THEN
            IF MAKE_PAYMENT(appointmentId) THEN
                PREPARE_FINAL_REPORT(appointmentId)
                DELIVER_REPORT_TO_MANAGER(appointmentId)
                BREAK
            ELSE
                LOG("Payment failed")
            END IF
        ELSE
            PROVIDE_FEEDBACK_TO_CCO()
        END IF
    END IF
END WHILE
```

END PROCEDURE

FUNCTION CCO_REVIEW(documents) RETURN CHECK_DOCUMENT_VALIDITY(documents) END FUNCTION

FUNCTION MANAGER_APPROVES(documents) RETURN VALIDATE_BY_MANAGER(documents) END FUNCTION

END FUNCTION

10.5.IT23540026 – Vidanapathirana I.D

Complaints and Feedback management

In this project, I developed the **Feedback and Complaints Management** component. The system allows students and staff to submit complaints or feedback digitally, ensuring transparency and accountability.

Key features include:

- **Complaint Registration**: Users can submit complaints with categories, descriptions, and optional attachments.
- **Feedback Submission**: Users provide feedback on services or staff behavior.
- **Admin Panel**: Admins can view, filter, and reply to complaints/feedback in real time.
- **Automated Status Updates**: Each complaint has a status (Pending, In Progress, Resolved) that updates automatically when handled.
- **PDF Report Generation**: Admin can export all complaints and feedback records into PDF format for auditing.
- **Data Visualization**: Pie charts and bar charts summarize complaints by category.

These features improve communication between users and administrators, while also ensuring that issues are tracked and resolved efficiently.

Completion Level of the Features

	Completion level of features	
	Features	Progress
Completed	Complaint Submission (User side CRUD: Add/Edit/Delete/Update	100%
	Complaint Management (Admin side CRUD: Add/Edit/Delete/Update)	100%
	Feedback Submission (User side CRUD: Add/Edit/Delete/Update)	100%
	Feedback Management (Admin side CRUD: Add/Edit/Delete/Update)	100%

	Admin View of Complaints & Feedback	100%
	Feedback/Complaint Resolution & Status Tracking	100%
	Frontend UI Implementation	100%
	Data Visualization (Pie & Bar Charts)	100%
	Filtering & Search Options	100%
	Real-Time Status Updates	100%
To be completed (Working on)	PDF Report Generation (Complaints & Feedback)	Not Completed
	Email / Notification System	Not Completed

Any SQL Queries Used

No, because MongoDB (via Mongoose) was used as the database, which is a NoSQL database system.

10.5, Algorithms, Flow Charts, Pseudocodes

Feedback System

This system is designed for open feedback, allowing users and admins to interact.

Algorithm for Feedback System

1. Initiation:

- The process starts when a customer's session is marked "Completed."
- The system automatically sends an SMS/email with a secure link to the feedback form related to the cart or service used during the session.

2. User (Customer) Actions:

- **Submit:**

    - The user clicks the link, views the feedback form for the items in their cart, and provides a rating (e.g., 1-5 stars) and comment for each item or service.
    - The user submits feedback for each item in the cart.
- **View:**
    - The user can browse all public feedback entries in a dedicated section or on the specific items they rated.
- **Edit/Delete:**
    - The user can modify or remove their own submitted feedback (for specific items) after verifying ownership through an authentication link.
- **Reply:**
    - The user can post a public reply to any feedback entry, including responses related to specific products in the cart.

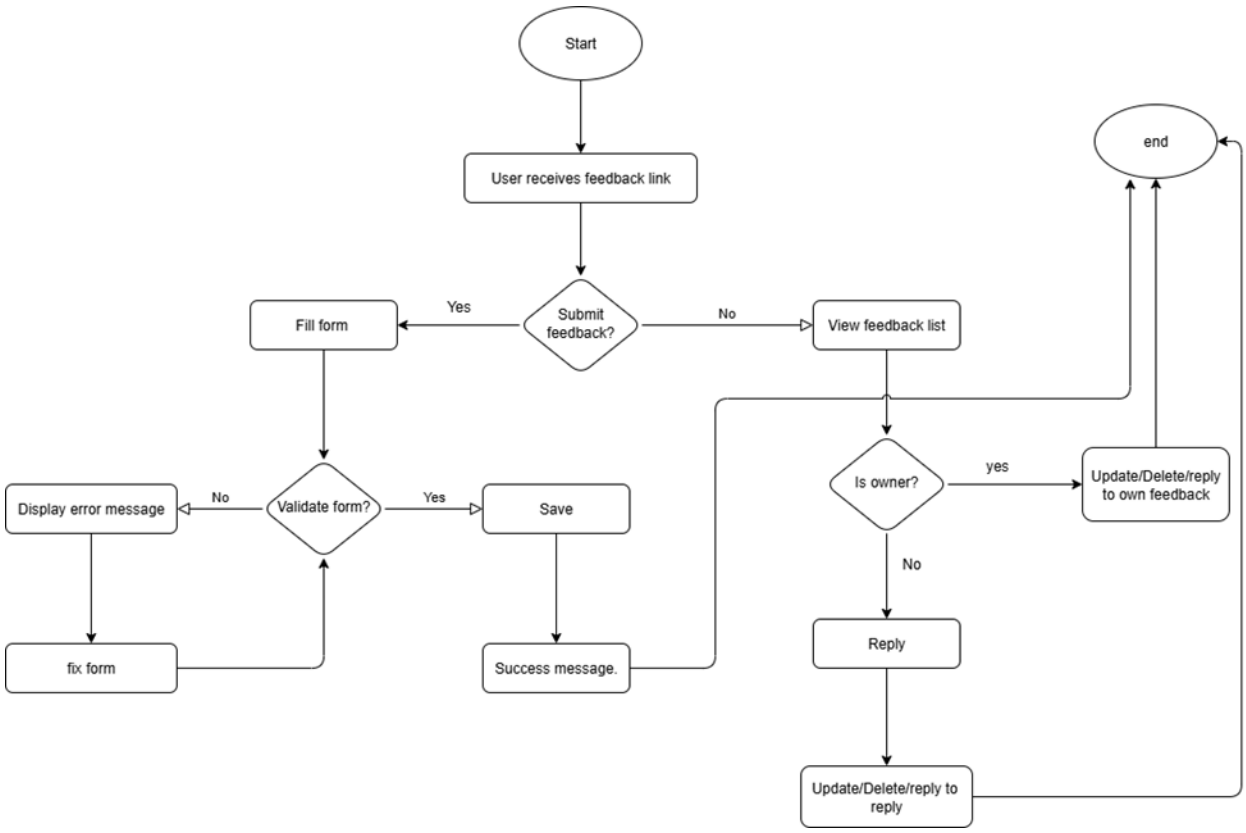
3. Admin Actions:

- **View:**
    - The admin can view all feedback and replies on a central dashboard, including feedback specifically tied to products in the cart.
- **Reply to Feedback:**
    - The admin selects any feedback entry related to cart items and posts an official reply, which is clearly marked with an "Admin" or "Official Response" tag.
- **Moderate:**
    - The admin has permission to delete any feedback or any reply submitted by any user. Admins can also edit their own replies.
    - Admins can also view aggregate cart analysis based on user feedback for specific items (e.g., average rating per product, common complaints, etc.).

4. Cart Analysis:

- **Analyze Feedback for Cart Items:**
    - For each product or service in the cart, the system tracks the feedback provided and aggregates the ratings (e.g., average rating, most common comment themes).
- **Generate Reports:**
    - The system generates reports for each product in the cart, analyzing user satisfaction based on feedback ratings and comments.
    - Admins can filter reports by category, rating, or time period.
    - Provide insights for improvement, such as common issues (e.g., if a product has consistent complaints about quality).

Flow Charts



Pseudocode for Feedback Management

User Feedback Management

```
// --- User submits feedback ---

FUNCTION SubmitFeedback(currentUser, formData)
   IF formData.rating IS EMPTY OR formData.comment IS EMPTY THEN
      RETURN "Error: Missing required fields."
   END IF

   newFeedback = CREATE Feedback(
      author = currentUser.ID,
      rating = formData.rating,
```

```
        comment = formData.comment
    )

    SAVE newFeedback TO Database
    RETURN "Success: Feedback submitted."
END FUNCTION



// --- User edits own feedback ---


FUNCTION EditFeedback(currentUser, feedbackID, newData)
    targetFeedback = FIND Feedback BY ID feedbackID

    IF targetFeedback IS NULL THEN
        RETURN "Error: Feedback not found."
    END IF

    IF targetFeedback.author ≠ currentUser.ID THEN
        RETURN "Error: Permission denied."
    END IF

    IF newData.rating IS NOT EMPTY THEN
        targetFeedback.rating = newData.rating
    END IF
    IF newData.comment IS NOT EMPTY THEN
        targetFeedback.comment = newData.comment
```

```
    END IF

    SAVE targetFeedback TO Database
    RETURN "Success: Feedback updated."
END FUNCTION


// --- User deletes own feedback ---

FUNCTION DeleteFeedback(currentUser, feedbackID)
    targetFeedback = FIND Feedback BY ID feedbackID

    IF targetFeedback IS NULL THEN
        RETURN "Error: Feedback not found."
    END IF

    IF targetFeedback.author ≠ currentUser.ID THEN
        RETURN "Error: Permission denied."
    END IF

    REMOVE targetFeedback FROM Database
    RETURN "Success: Feedback deleted."
END FUNCTION


// --- User replies to any feedback (own or others) ---
```

```
FUNCTION ReplyToFeedback(currentUser, feedbackID, replyText)
    targetFeedback = FIND Feedback BY ID feedbackID

    IF targetFeedback IS NULL THEN
        RETURN "Error: Feedback not found."
    END IF

    newReply = CREATE Reply(
        author = currentUser.ID,
        text = replyText,
        timestamp = CURRENT_TIME
    )

    ADD newReply TO targetFeedback.replies
    SAVE targetFeedback TO Database
    RETURN "Success: Reply posted."
END FUNCTION
```

Complaint Management System

This system is a formal process for submitting and resolving specific issues.

Algorithm for Complaint Management System

1. **Submission:**

- A user logs in and navigates to the "Submit a Complaint" page.
- They fill out a detailed form, including the complaint category and description.
- The system saves the complaint with a status of **"Pending"** and generates a unique **Complaint ID**.
- The user receives this ID for tracking.

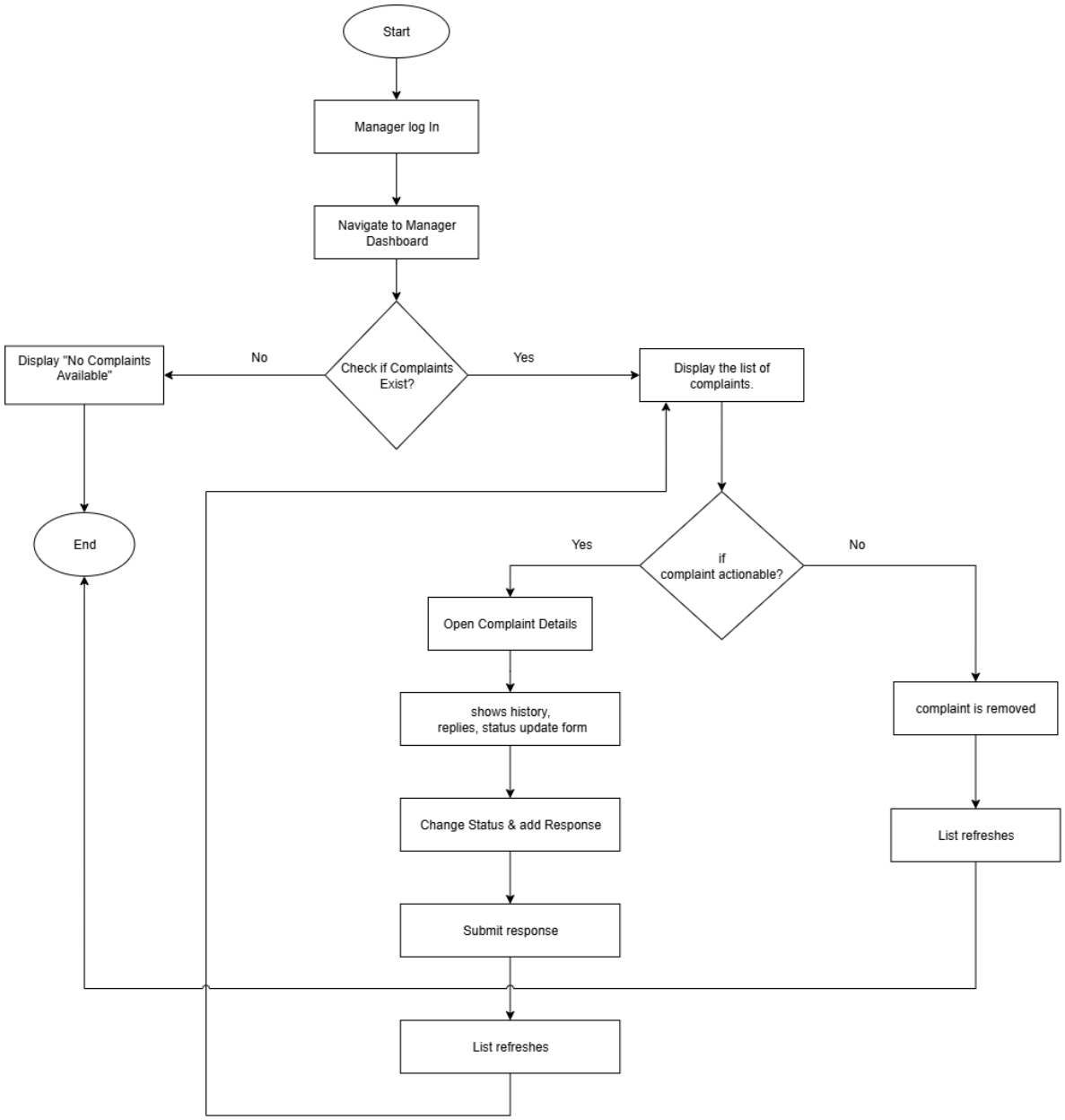
2. **Tracking:**

- The user can view the real-time status of their submitted complaints ("Pending," "In Progress," "Resolved") on their personal dashboard.
- **Analysis:** Track the complaint status distribution over time (how many are pending, in progress, or resolved).

3. **Admin Management:**

- An Admin views a list of new complaints on their dashboard.
- They review a complaint and update its status to **"In Progress"**.
- Once the issue is resolved, they update the status to **"Resolved"** and add internal resolution notes.
- The status update is automatically visible to the customer.

Flow Charts



FUNCTION ManageComplaints(admin):

```
// Step 1: Manager Logs In
IF admin IS logged IN:

    // Step 2: Navigate to Manager Dashboard
    Navigate_To_Manager_Dashboard()

    // Step 3: Check if Complaints Exist
    IF Check_Complaints_Exist() == NO:
        DISPLAY "No Complaints Available"
        END
    ELSE:
        DISPLAY list of complaints

        // Step 4: Check if Complaint is Actionable
        FOR each complaint IN complaints:
            IF complaint IS actionable:
                // Step 5: Open Complaint Details
                OPEN complaint details
                SHOW history, replies, and status update form

                // Step 6: Change Status and Add Response
                INPUT newStatus FROM ["In Progress", "Resolved"]
                UPDATE complaint status = newStatus

                // Step 7: If Resolved, Add Internal Resolution Notes
                IF newStatus == "Resolved":
                    INPUT internalNotes
                    UPDATE complaint internalNotes = internalNotes

                // Step 8: Submit Response
                SUBMIT response

                // Step 9: Refresh Complaint List
                REFRESH complaint list
            ELSE:
                // Step 10: If Not Actionable, Remove Complaint
                REMOVE complaint
                REFRESH complaint list

    ELSE:

        DISPLAY "Admin login required"
```

```
END FUNCTION
```